
Modül 12 — Spring Security ile Güvenlik

Ödevler

Spring Boot Edu



2026-09-24

İçindekiler

Nasıl Çalışılır	2
Ödev 1 — Rol Bazlı Erişim (Kolay)	2
Ödev 2 — Yalnızca Kendi Siparişlerin (Orta)	3
Ödev 3 — JWT Claim'lerinden Roller (Zor)	3
Ek Meydan Okuma (İsteğe Bağlı)	4

Nasıl Çalışılır

1. Başlangıç kodu modules/12-security/exercise/ altındadır. TODO yorumlarını bulun.
2. Ödevler Docker istemez: Kullanıcılar bellekte durur ve testler kendi JWT'lerini imzalar.
3. Testleri çalıştırın:

```
./mvnw -Pexercises -pl modules/12-security/exercise -am test
```

4. Tüm testler yeşil olduğunda ödev tamamdır.
5. Takılırsanız önce ipuçlarını, sonra modules/12-security/solution/ altındaki çözümü okuyun.

★ İpucu

given paketi kullanıcıları (CUSTOMER olarak ada, bob; ADMIN olarak admin), API controller'larını ve bir imzalama anahtarını içerir. Onu değiştirmeniz gerekmez.

Ödev 1 — Rol Bazlı Erişim (Kolay)

Hedef: Mağaza API'sini iki rol için URL kurallarıyla korumak.

İstek	Kim çağırabilir
GET /api/catalog	herkes, giriş yapmadan da
POST /api/catalog	ADMIN
/api/cart/**	CUSTOMER
/api/admin/**	ADMIN
geri kalan her şey	giriş yapmış her kullanıcı

Yapılacaklar (exercisel.ExerciseSecurityConfiguration):

- TODO 1a — GET /api/catalog herkese açık. POST /api/catalog yalnızca ADMIN için.
- TODO 1b — /api/cart/** yalnızca CUSTOMER için.
- TODO 1c — /api/admin/** yalnızca ADMIN için.

İpuçları:

- Ders bölüm 3.3: `.requestMatchers(HttpMethod.GET, "/api/catalog").permitAll(), .hasRole("ADMIN")`.
- Kurallar **sırayla** kontrol edilir ve eşleşen ilk kural kazanır. `anyRequest()` en sonda kalmalıdır.

- Bir admin otomatik olarak customer değildir: `hasRole("CUSTOMER")` admin'i reddeder.

Kabul kriterleri: Exercise1Test içindeki 5 testin hepsi geçer.

Tahmini süre: 20 dakika

Ödev 2 — Yalnızca Kendi Siparişlerin (Orta)

Hedef: Müşteriler yalnızca kendi siparişlerini, admin'ler hepsini görür. Kurallar, her çağırana için geçerli olsun diye servise aittir.

Yapılacaklar (exercise2.OrderQueries):

- TODO 2a — `ordersOf(customer)`: Yalnızca müşterinin kendisi veya bir ADMIN çağırabilir.
- TODO 2b — `recentOrders()`: Döndürülen listeden çağırana ait olmayan her siparişi çıkarın.
- TODO 2c — `find(id)`: Döndürülen siparişi yalnızca sahibi olan müşteri veya bir ADMIN görebilir.

İpuçları:

- Ders bölüm 3.4: `@PreAuthorize`, `@PostAuthorize` ve `returnObject`.
- Bir metod parametresine ifadeye `#` ile erişilir: `#customer == authentication.name`.
- `@PostFilter("filterObject.customer() == authentication.name")` döndürülen koleksiyondan eleman çıkarır. Değiştirilebilir bir koleksiyon ister; `recentOrders()` zaten öyle bir koleksiyon döndürür.

Kabul kriterleri: Exercise2Test içindeki 5 testin hepsi geçer.

Tahmini süre: 30 dakika

Ödev 3 — JWT Claim'lerinden Roller (Zor)

Hedef: Kimlik sağlayıcımız kullanıcının rollerini kendi claim'ine, okunabilir kullanıcı adını ise `preferred_username`'e koyar:

```
{ "sub": "user-7", "preferred_username": "ada", "roles": ["customer"], "iss":  
  "https://id.bookstore.example" }
```

Spring Security varsayılan olarak adı `sub`'dan, yetkileri `scope`'tan alır. Ödev 1'in kuralları bu token'larla çalışsın diye claim'leri eşleyin.

Yapılacaklar (exercise3.JwtRolesConfiguration):

- TODO 3a — Bir `JwtAuthenticationConverter` bean'i. Resource server onu otomatik kullanır.
- TODO 3b — Principal adı `preferred_username` claim'inden gelir.
- TODO 3c — `roles` claim'indeki her değer `ROLE_<BÜYÜK HARFLE ROL>` yetkisine dönüşür, ör. `customer` → `ROLE_CUSTOMER`.

İpuçları:

- `converter.setPrincipalClaimName(...)`.
- `converter.setJwtGrantedAuthoritiesConverter(jwt -> ...)` Jwt'yi alır. Claim'i olmayan bir token için `jwt.getClaimAsStringList("roles")` null olabilir.
- `role.toUpperCase(Locale.ROOT)`: Sabit bir locale olmadan Türkçe bir sistemde "admin".toUpperCase() sonucu "ADMİN" (noktalı İ) olur ve rol kontrolü başarısız olur.

Kabul kriterleri: Exercise3Test içindeki 3 testin hepsi geçer.

Tahmini süre: 40 dakika

Ek Meydan Okuma (İsteğe Bağlı)

Ödev 3'ün JWT'leri yalnızca imza ve süre açısından kontrol edilir. Token'lar yalnızca `https://id.bookstore.example issuer`'ından gelirse ve `aud` claim'i `bookstore-api` içerirse kabul edilsin diye validator'lar ekleyin (`JwtValidators.createDefaultWithValidators(...)`, `JwtClaimValidator`). Başka bir issuer'dan gelen token için bir test yazın.